

Intelligent Control

Session 4: When the Model Meets Reality

Dr. Gerardo Flores

SENG 5360 · Texas A&M International University ·

Today

1. What if the model is wrong? — *parametric uncertainty*
2. What if something pushes the system? — *external disturbances*
3. What if the sensor is noisy? — *measurement noise*
4. Putting it all together: the pendulum on a cart
5. Simulation lab

Learning objectives

- predict when the controller of Session 3 stops working on a heavier pendulum;
- compute the offset left by a constant push, and remove it with integral action;
- explain why differentiating a noisy signal is dangerous, and filter it;
- linearize, check and stabilize a four-state system: the pendulum on a cart.

Before we start: get the code

Every example and exercise of today, except the final cart-pole example, runs from **one** Python file:

`pendulum_inverted.py`

1. Go to the course repository on GitHub: `github.com/gfloresc/intelligent-control`
2. Open the folder `code/` and download `pendulum_inverted.py`.
3. Open it in Google Colab (or any Python installation with `numpy` and `matplotlib`) and run it. You should see the pendulum swing back to the top.

How to use it. Near the top of the file there is a list called `SCENARIOS`, with one entry for each example of the slides. To run a different one, change a single line,

`SCENARIO = 'design'` $\longrightarrow$ `SCENARIO = 'push_0.2',`

and run again. You never need to edit anything else.

In `pendulum_inverted.py`, mark `[Q1] ...` `[FIX-F]`: throughout the slides, a grey box like this one tells you which scenario to run and which lines to look at. Search the file for the mark to find them.

Where we left off

In Session 3 we balanced the pendulum upright with

$$u = -k_1\,(\theta - \pi) - k_2\,\dot{\theta}, \qquad k_1 = 3.0915, \quad k_2 = 0.3280,$$

and it worked. But it worked in a *perfect* world: the simulation used exactly the mass we designed for, nothing pushed the pendulum, and the controller knew θ and $\dot{\theta}$ exactly.

Session 1 already warned us that the real world adds three things to the model:

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u}, \boldsymbol{\vartheta}) + \mathbf{d}, \qquad \mathbf{y} = \mathbf{x} + \boldsymbol{\eta}.$$

Term	Name	Pendulum example
$\boldsymbol{\vartheta}$	parametric uncertainty	the real mass is not 0.5 kg
$\mathbf{d}$	external disturbance	wind, a hand, an unmodelled torque
$\boldsymbol{\eta}$	measurement noise	the encoder reading jitters

Today we switch them on one at a time and ask a simple question each time: *does the controller still work?*

Question 1: what if the model is wrong?

We designed k_1, k_2 for $m = 0.5\,\text{kg}$. Suppose the pendulum actually carries a heavier load, and we *do not change the gains*.

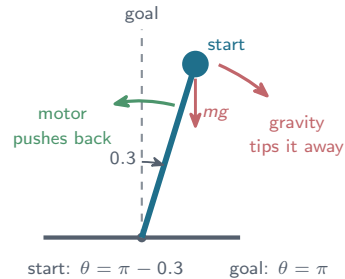
The test. The pendulum starts 0.3 rad (about 17°) away from upright, at rest, and the controller is switched on.

What we want to see. The pendulum swings back to the vertical dashed line and stays there, as it did in Session 3.

Who is fighting whom. Two torques act on the rod:

- gravity pulls the mass down, so it tips the rod further away from the top;
- the motor pushes the rod back toward the top, with a torque that grows with the tilt: $k_1\,\delta\theta$, where $\delta\theta = \theta - \pi$ is the tilt (zero at the top).

A heavier mass makes gravity stronger, but the motor gains stay the same. Does the motor still win?



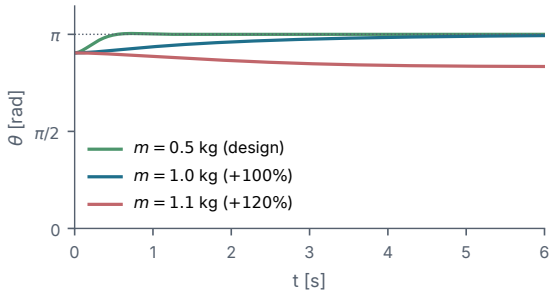
Question 1: what actually happens

The same controller on three pendulums, all starting 0.3 rad away from upright:

- $m = 0.5$ kg: the design case, back up in about 1 s;
- $m = 1.0$ kg: it still returns, but very slowly;
- $m = 1.1$ kg: it **never** gets back up; it settles about 0.52 rad below upright.

So the controller tolerates a large error in the mass, but not an unlimited one. Where exactly is the limit? The next slide answers that with one inequality.

In `pendulum_inverted.py`, mark `[Q1]`: `M_REAL` in `plant()` is the real mass; the gains never see it. Run `SCENARIO = 'design', 'heavy', 'too_heavy'`.



The rule, step by step: the torques near the top

We want to know exactly when the pendulum gets back up. We follow four steps.

Step 1. Write the model with the *real* mass m (the plant does not care what mass we designed for):

$$m\ell^2\,\ddot{\theta} = -mg\ell\sin\theta - b\,\dot{\theta} + u.$$

Step 2. Look near the top. Call the tilt $\delta\theta = \theta - \pi$, so $\theta = \pi + \delta\theta$. Since $\sin(\pi + \delta\theta) = -\sin\delta\theta \approx -\delta\theta$ for a small tilt, the gravity term becomes

$$-mg\ell\sin\theta \approx +mg\ell\,\delta\theta.$$

The sign is the important part: the gravity torque has the *same* sign as the tilt, so it pushes the pendulum further **away** from the top. Also, $\ddot{\theta} = \delta\ddot{\theta}$ and $\dot{\theta} = \delta\dot{\theta}$, because π is a constant.

Step 3. Put in the controller of Session 3, $u = -k_1\,\delta\theta - k_2\,\delta\dot{\theta}$. Its torque has the *opposite* sign of the tilt, so it pushes **back**:

$$m\ell^2\,\delta\ddot{\theta} = \underbrace{mg\ell\,\delta\theta}_{\text{gravity: away}} - b\,\delta\dot{\theta} \underbrace{- k_1\,\delta\theta - k_2\,\delta\dot{\theta}}_{\text{motor: back}}.$$

Step 4. Move everything to the left and group the terms with the same variable:

$$\boxed{m\ell^2\,\delta\ddot{\theta} + (b + k_2)\,\delta\dot{\theta} + (k_1 - mg\ell)\,\delta\theta = 0}$$

The next slide reads this equation.

The rule: the controller must beat gravity

Compare the boxed equation with a mass on a spring and a damper, which you know from physics:

$$M\ddot{x} + c\,\dot{x} + k\,x = 0 \qquad \longleftrightarrow \qquad \underbrace{m\ell^2}_{M}\,\delta\ddot{\theta} + \underbrace{(b + k_2)}_{c}\,\delta\dot{\theta} + \underbrace{(k_1 - mg\ell)}_{k}\,\delta\theta = 0.$$

- If the spring constant k is **positive**, the spring pulls the mass back to $x = 0$: the pendulum returns to the top.
- If k is **negative**, the "spring" pushes the mass away: the pendulum falls.

The damper $c = b + k_2 = 0.378$ is positive, so it only slows the motion down. Hence the pendulum is held upright if and only if

$$\boxed{k_1 - mg\ell > 0 \quad \Longleftrightarrow \quad k_1 > mg\ell}$$

In words: the motor gain must beat the gravity torque, which grows with the mass.

m [kg]	$mg\ell$	spring $k_1 - mg\ell$	eigenvalues	behaviour
0.5	1.47	$+1.62$	$-4.20 \pm j4.29$	returns in about 1 s
1.0	2.94	$+0.15$	$-3.76,\ -0.44$	returns, slowly (weak spring)
1.1	3.24	-0.15	$-4.17,\ +0.35$	does not return

The eigenvalues are the roots of $m\ell^2\lambda^2 + (b + k_2)\lambda + (k_1 - mg\ell) = 0$ (Session 2). As the spring weakens, one of them moves toward zero (slower return); when the spring turns negative, it becomes positive (no return). This is the figure of Question 1.

Exercise 1

Use the rule $k_1 > mg\ell$, with $g = 9.81\,\mathrm{m/s^2}$ and $\ell = 0.3\,\mathrm{m}$.

1. With our gain $k_1 = 3.0915$, what is the largest mass the controller can hold upright?
2. A customer says the pendulum may carry up to $2\,\mathrm{kg}$. What is the smallest k_1 that still works?

Hint

Both parts are the same inequality, solved once for m and once for k_1.

Exercise 1: solution

Step 1. Solve the rule for the mass:

$$m < \frac{k_1}{g\ell} = \frac{3.0915}{9.81 \times 0.3} = \frac{3.0915}{2.943} \approx 1.05\,\text{kg}.$$

The design mass is 0.5 kg, so the controller survives a mass error of about 110 %. This matches the figure: 1.0 kg works, 1.1 kg does not.

Step 2. Solve the rule for the gain, with $m = 2\,\text{kg}$:

$$k_1 > mg\ell = 2 \times 9.81 \times 0.3 = 5.886.$$

Step 3. In practice we would not choose $k_1 = 5.9$, because then the heaviest load would settle very slowly, exactly like the 1.0 kg case before. We leave margin and choose something clearly larger, for example $k_1 = 8$ or more.

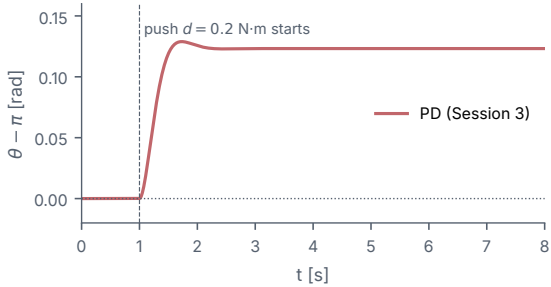
Question 2: what if something pushes?

Now the mass is right ($m = 0.5\,\text{kg}$), and the pendulum starts exactly upright. At $t = 1\,\text{s}$ a constant torque $d = 0.2\,\text{N m}$ starts pushing it, like a steady wind. The controller is the PD of Session 3, and it does *not* know about d.

What we want to see. The pendulum leans a little when the push starts, then the controller brings it back to $\theta = \pi$.

What actually happens (figure). The pendulum leans and **stays** leaning: it settles at $\theta - \pi = 0.123\,\text{rad}$, about 7°, and never comes back. This permanent error is called the **offset**, or steady-state error, written $\delta\theta_{ss}$.

Where does the number 0.123 come from? The next slide computes it.



In `pendulum_inverted.py`, mark `[Q2]`: `push(t)` is $d(t)$; it is added to u inside `plant()`. Run `'push_0.1'` and `'push_0.2'`.

Where the offset comes from

Step 1. Take the boxed equation of the rule and add the push. The push d is a torque, so it enters exactly like the motor torque:

$$m\ell^2\,\delta\ddot{\theta} + (b + k_2)\,\delta\dot{\theta} + (k_1 - mg\ell)\,\delta\theta = d.$$

Step 2. Wait until the pendulum stops moving. At rest the velocity and the acceleration are zero, $\delta\dot{\theta} = 0$ and $\delta\ddot{\theta} = 0$, so the first two terms disappear:

$$(k_1 - mg\ell)\,\delta\theta_{ss} = d.$$

Step 3. Solve for the tilt:

$$\boxed{\delta\theta_{ss} = \frac{d}{k_1 - mg\ell}} \qquad \Longrightarrow \qquad \delta\theta_{ss} = \frac{0.2}{3.0915 - 1.4715} = \frac{0.2}{1.62} \approx 0.123\,\text{rad},$$

which is exactly what the simulation shows.

Why it cannot be zero. In Step 2, if $\delta\theta_{ss}$ were zero, the left side would be zero but the right side is $d \neq 0$. The PD only produces torque when there is a tilt, so it *needs* a permanent tilt to cancel a permanent push. It is Hooke's law, $x = F/k$: a stiffer spring (larger k_1) stretches less, but it always stretches.

Exercise 2

Use $\delta\theta_{ss} = d/(k_1 - mg\ell)$ with $mg\ell = 1.4715$.

1. With $k_1 = 3.0915$, what offset does a push of $d = 0.1\,\text{N m}$ leave? Give it in radians and in degrees.
2. We want the offset for $d = 0.2\,\text{N m}$ to be at most $0.02\,\text{rad}$. What k_1 do we need?
3. Is raising the gain a good way to fix the offset?

Exercise 2: solution

Step 1. The spring constant of the loop is $k_1 - mg\ell = 3.0915 - 1.4715 = 1.62$.

Step 2. Offset for $d = 0.1\,\mathrm{N\,m}$:

$$\delta\theta_{ss} = \frac{0.1}{1.62} \approx 0.062\,\mathrm{rad} \approx 3.5^\circ.$$

Step 3. Solve for k_1 with $d = 0.2$ and $\delta\theta_{ss} = 0.02$:

$$k_1 = mg\ell + \frac{d}{\delta\theta_{ss}} = 1.4715 + \frac{0.2}{0.02} \approx 11.5.$$

Step 4. That is almost four times the original gain, and it only makes the offset smaller, never zero. A larger gain also asks the motor for more torque and multiplies any sensor noise by a larger number. Raising the gain is a poor fix.

The practical fix: integral action

The PD reacts only to the tilt *now*. We add a third term that reacts to the tilt *accumulated* over time:

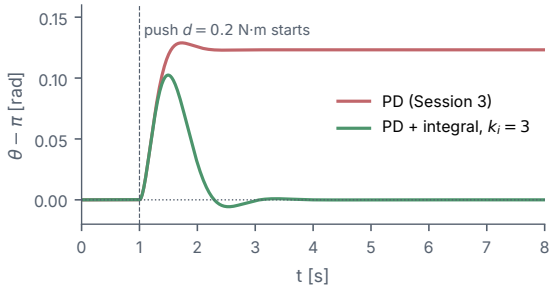
$$u = -k_1\,\delta\theta - k_2\,\dot{\theta} - k_i\,z, \qquad z(t) = \int_0^t \delta\theta(s)\,ds.$$

z is the area under the error curve, and k_i is the **integral gain**.

How it behaves. While the pendulum is tilted, z keeps growing, so the term $-k_i z$ keeps pushing harder. It only stops growing when the tilt is zero.

In the figure, $k_i = 3$: the pendulum leans when the push starts, and about 2 s later it is back at exactly π.

This is the **I** of the PID controller used in industry.



In `pendulum_inverted.py`, **mark** `[FIX-I]`: the two lines of `controller()`: first z, then u. Run `'integral'`, then `'integral_too_much'`.

Why integral action removes the offset

We repeat the steps of the offset slide, now with the integral term.

Step 1. The closed loop has one more state, z, with $\dot{z} = \delta\theta$:

$$m\ell^2\,\delta\ddot{\theta} + (b + k_2)\,\delta\dot{\theta} + (k_1 - mg\ell)\,\delta\theta + k_i\,z = d, \qquad \dot{z} = \delta\theta.$$

Step 2. At rest *nothing* changes, so also $\dot{z} = 0$. But $\dot{z} = \delta\theta$, hence

$$\boxed{\delta\theta_{ss} = 0}$$

The offset is forced to be zero: if any tilt remained, z would still be changing.

Step 3. Put $\delta\theta_{ss} = 0$ back into the first equation. All terms vanish except

$$k_i\,z_{ss} = d \qquad \Longrightarrow \qquad u_{ss} = -k_i\,z_{ss} = -d = -0.2\,\mathrm{N\,m}.$$

The integral term has learned the push by itself and cancels it exactly. In the animation you can see u settle at $-0.2\,\mathrm{N\,m}$.

A word of caution. Too much integral gain makes the loop oscillate: here the swings *grow* for k_i above about 13.6 (scenario `'integral_too_much'`, $k_i = 15$). Start with a small k_i.

Question 3: what if the sensor is noisy?

So far the controller knew θ and $\dot{\theta}$ exactly. A real pendulum is different.

What we measure. An **encoder** on the pivot reads the angle every T_s seconds (here $T_s = 0.01\,\text{s}$). Each reading has a small random error η_k, the **noise**, of typical size $\sigma_\eta = 0.01\,\text{rad}$ (about 0.6°):

$$y_k = \theta_k + \eta_k, \qquad k = 0, 1, 2, \ldots$$

The controller never sees the true angle θ_k, only the reading y_k.

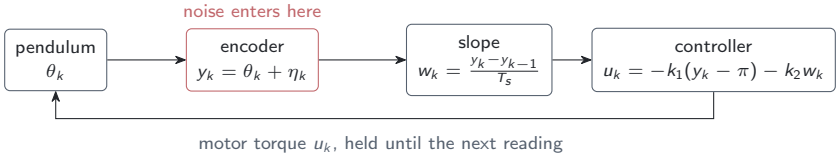
What we do not measure. Nothing measures the velocity $\dot{\theta}$, yet the controller needs it for the term $-k_2\,\dot{\theta}$. The obvious idea is the slope between two readings:

$$w_k = \frac{y_k - y_{k-1}}{T_s} \approx \dot{\theta}.$$

Is that a problem? To see it, we follow the noise from the encoder to the motor.

The chain from the pendulum to the motor

Every T_s seconds the controller repeats the same chain of operations:



1. the pendulum has its true angle θ_k;
2. the encoder adds the noise and returns y_k; this is the *only* place where noise enters;
3. the controller computes the slope w_k from the last two readings;
4. it computes the torque u_k from y_k and w_k, and the motor applies it until the next reading.

The noise η_k is small, about 0.6°. The next slide shows what step 3 does to it.

In `pendulum_inverted.py`, mark `[Q3]`: y_k is made in `sensor()`, the controller receives only y_k, and w_k is the first line of `velocity()`. Run `'noise_raw'`, then `'noise_fast'`.

Why the slope amplifies the noise

Step 1. Put $y_k = \theta_k + \eta_k$ into the slope and separate the two parts:

$$w_k = \frac{y_k - y_{k-1}}{T_s} = \underbrace{\frac{\theta_k - \theta_{k-1}}{T_s}}_{\approx\, \dot{\theta} \text{ (what we want)}} + \underbrace{\frac{\eta_k - \eta_{k-1}}{T_s}}_{\text{noise part}}.$$

Step 2. Size of the noise part. Each η is about $\pm\sigma_\eta$, and the two are independent, so in a bad case they have opposite signs and their difference is about $2\sigma_\eta$. Dividing by T_s:

$$\text{velocity error} \approx \boxed{\frac{2\sigma_\eta}{T_s}}$$

Step 3. Why this is large. The true angle changes very little in 0.01 s, so the first part is a small number divided by a small number: fine. The noise does *not* shrink when T_s shrinks, so the second part is a fixed number divided by a small number: large. With our values, $2 \times 0.01/0.01 = 2$ rad/s.

From the velocity error to the motor

Step 4. The controller multiplies the velocity error by k_2 and sends it to the motor:

$$\text{torque noise} \approx k_2 \, \frac{2\sigma_\eta}{T_s}.$$

The surprise

Sampling *faster* makes it *worse*: a smaller T_s divides the same noise by a smaller number. This is the opposite of what most people expect.

Notice that the angle reading itself is harmless: its noise reaches the motor as $k_1\sigma_\eta \approx 3 \times 0.01 = 0.03\,\text{N m}$, which is negligible. The problem is only the velocity.

Exercise 3

The encoder noise is $\sigma_\eta = 0.01\,\text{rad}$ (about 0.6°) and the controller samples every $T_s = 0.01\,\text{s}$. Use $k_2 = 0.328$.

1. Estimate the error in $\dot{\theta}$ caused by the noise.
2. How much torque noise does that put on the motor? Compare it with $mg\ell = 1.47\,\text{N m}$, the largest torque gravity can apply.
3. What happens if we sample ten times faster, $T_s = 0.001\,\text{s}$?

Exercise 3: solution

Step 1. Velocity error:

$$\frac{2\sigma_\eta}{T_s} = \frac{2 \times 0.01}{0.01} = 2\,\text{rad/s}.$$

Step 2. Torque noise:

$$k_2 \times 2 = 0.328 \times 2 \approx 0.66\,\text{N m},$$

which is about 45 % of $mg\ell$. The motor is shaking with almost half the torque that gravity applies, just because of a 0.6° jitter in the sensor.

Step 3. Ten times faster sampling:

$$\frac{2 \times 0.01}{0.001} = 20\,\text{rad/s} \quad \Longrightarrow \quad 0.328 \times 20 \approx 6.6\,\text{N m},$$

more than four times the gravity torque. Faster sampling, much worse noise.

The practical fix: a low-pass filter

Idea: the true velocity changes slowly, the noise jumps randomly up and down. If we *average* the recent slopes, the random jumps cancel and the slow part survives.

Step 1. Keep a running value $\hat{w}_k$, the **filtered velocity**, and start it at $\hat{w} = 0$.

Step 2. At every reading, move $\hat{w}$ only a small fraction α of the way toward the new raw slope w_k:

$$\hat{w}_k = \hat{w}_{k-1} + \alpha \left(w_k - \hat{w}_{k-1} \right), \qquad \alpha = \frac{T_s}{\tau}.$$

The next slide shows why this removes the noise.

How the filter removes the noise

Step 3. Rewrite it to see that it is a weighted average of the old value and the new reading:

$$\hat{w}_k = (1 - \alpha)\,\hat{w}_{k-1} + \alpha\, w_k .$$

With $\tau = 0.05\,\text{s}$ and $T_s = 0.01\,\text{s}$, $\alpha = 0.2$: each new reading counts 20 % and the history counts 80 %. A single noisy reading can only move $\hat{w}$ a little.

Step 4. The controller uses $\hat{w}_k$ instead of w_k:

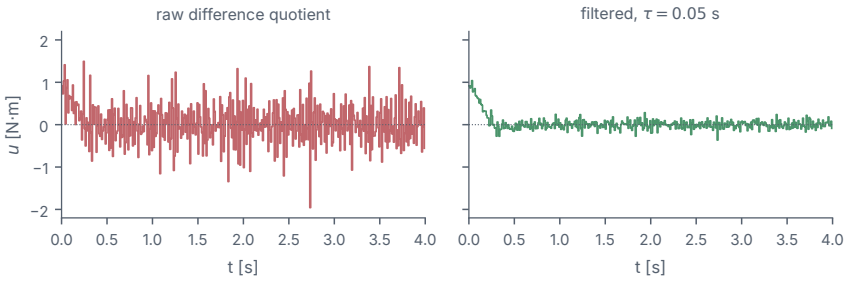
$$u_k = -k_1\,(y_k - \pi) - k_2\,\hat{w}_k .$$

The number τ is the **filter time constant**: roughly, the filter averages over the last τ seconds. Larger τ, smoother but slower.

In `pendulum_inverted.py`, mark `[FIX-F]`: one line of `velocity()`, applied only when the scenario sets `tau`. Run `'noise_raw'`, then `'noise_filtered'`.

The filter at work

The torque u sent to the motor, same pendulum, same noise, without and with the filter:



- **Left, no filter.** u jumps by about 0.5 N m at every reading. The pendulum still balances, but a real motor would vibrate, heat up and wear out.
- **Right, $\tau = 0.05$ s.** The jitter drops to about 0.1 N m, five times less, and the pendulum still balances.

Filtering has a price

A filter averages over roughly the last τ seconds: that removes the noise, but it also makes the velocity estimate arrive *late*.

τ	noise on u	effect on the pendulum
off	large, ≈ 0.5 N m	balances, but the motor shakes
0.05 s	small, ≈ 0.1 N m	balances, smoothly
0.5 s	very small	overshoots by 0.12 rad, still swinging after 4 s

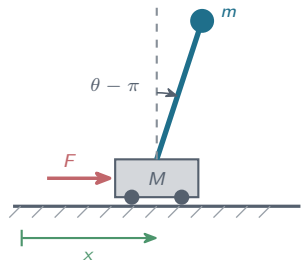
With $\tau = 0.5$ s the damping term $k_2\dot{\theta}$ reacts to where the pendulum *was*, not where it *is*, so the loop loses its damping. The rule of thumb: choose τ much smaller than the settling time of the loop. Here the loop settles in about 1 s: $\tau = 0.05$ s is 20 times smaller (fine), $\tau = 0.5$ s is only half of it (too slow).

In `pendulum_inverted.py`, mark `[FIX-F]`: the three rows of the table are `'noise_raw'`, `'noise_filtered'` and `'noise_slow_filter'`.

Looking ahead. The proper way to estimate $\dot{\theta}$ is an **observer**, which uses the model of the pendulum instead of a blind average. That is Module 3.

Putting it all together: the pendulum on a cart

Our pendulum ($m = 0.5\,\text{kg}$, $\ell = 0.3\,\text{m}$, $b = 0.05$) now sits on a cart of mass $M = 1\,\text{kg}$ that rolls on a track. This is the classic **cart-pole**.



What changes. The motor no longer turns the pendulum. It pushes the *cart* with a force F, and the pendulum is balanced only by moving the cart under it, like balancing a broom on your hand.

The goal. Hold the pendulum upright, $\theta = \pi$, *and* keep the cart at the center, $x = 0$.

The state has four entries and the input is the force:

$$\mathbf{x} = \begin{bmatrix} x & \dot{x} & \theta & \dot{\theta} \end{bmatrix}^\top, \qquad u = F.$$

As before, θ is measured from straight down.

We follow the steps of Module 1: model, equilibrium, linearization, design, and test on the nonlinear system.

In `cartpole.py`, mark `[CP-MODEL]`: the animation is in a separate file, `cartpole.py`, in the same `code/` folder. It works like `pendulum_inverted.py`.

Step 1: the nonlinear model (given)

Newton's laws for the cart and the pendulum give two coupled equations. We take them as given:

$$(M+m)\,\ddot{x} + m\ell\cos\theta\,\ddot{\theta} - m\ell\sin\theta\,\dot{\theta}^2 = F, \qquad \text{(cart)}$$

$$m\ell^2\,\ddot{\theta} + m\ell\cos\theta\,\ddot{x} + mg\ell\sin\theta + b\,\dot{\theta} = 0. \qquad \text{(pendulum)}$$

How to read them:

- the cart equation is mass $\times$ acceleration $=$ force, plus the terms that appear because the pendulum swings on top of it;
- the pendulum equation is the one of Sessions 1 to 4, plus the term $m\ell\cos\theta\,\ddot{x}$: **accelerating the cart tips the pendulum**. This term is the only way F reaches the pendulum.

To use the tools of Session 3 we need the state-space form $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, F)$, with $\mathbf{x} = (x_1, x_2, x_3, x_4) = (x, \dot{x}, \theta, \dot{\theta})$. Both equations contain $\ddot{x}$ and $\ddot{\theta}$, so we solve them as two linear equations in those two unknowns. The next slide gives the result.

In `cartpole.py`, mark `[CP-MODEL]`: `plant()` writes the two equations as comments and then the solved $\ddot{x}$, $\ddot{\theta}$.

Step 1 (continued): the model in state-space form

Solving the two equations for $\ddot{x}$ and $\ddot{\theta}$ gives the four components of $\mathbf{f}$:

$$
\begin{aligned}
f_1 &= \dot{x}, \\
f_2 &= \ddot{x} = \frac{N(\mathbf{x}, F)}{D(\theta)}, \qquad N = F + m\ell \sin\theta\, \dot{\theta}^2 + mg \sin\theta \cos\theta + \frac{b}{\ell} \cos\theta\, \dot{\theta}, \qquad D = M + m \sin^2\theta, \\
f_3 &= \dot{\theta}, \\
f_4 &= \ddot{\theta} = -\frac{1}{\ell}\Big(\cos\theta\, f_2 + g \sin\theta + \frac{b}{m\ell}\, \dot{\theta} \Big).
\end{aligned}
$$

Two things to notice before we differentiate:

- f_4 is just the pendulum equation divided by $m\ell^2$ and solved for $\ddot{\theta}$; it contains f_2 because $\ddot{x}$ appears in it;
- neither f_2 nor f_4 contains x or $\dot{x}$: the physics does not depend on *where* the cart is or how fast it rolls (we neglect cart friction).

This $\mathbf{f}$ is exactly what the simulation integrates.

Step 2: equilibrium and the shape of the Jacobians

Equilibrium. At rest ($\dot{x} = \dot{\theta} = 0$) with $F = 0$ we need $f_2 = 0$ and $f_4 = 0$. With $f_2 = 0$, $f_4 = 0$ reduces to $g \sin\theta = 0$, so $\sin\theta = 0$ (and then also $N = 0$). Upright: $\theta = \pi$, with the cart *anywhere*. We choose

$$\mathbf{x}^\star = (0,\ 0,\ \pi,\ 0), \qquad F^\star = 0.$$

Jacobians (Session 3): $\mathbf{A} = \partial\mathbf{f}/\partial\mathbf{x}$ and $\mathbf{B} = \partial\mathbf{f}/\partial F$, evaluated at $(\mathbf{x}^\star, F^\star)$; row i differentiates f_i, column j is with respect to the j-th state. Most entries can be filled in without computing anything:

$$\mathbf{A} = \left[\frac{\partial f_i}{\partial x_j}\right]_\star = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & a_{23} & a_{24} \\ 0 & 0 & 0 & 1 \\ 0 & 0 & a_{43} & a_{44} \end{bmatrix}, \qquad \mathbf{B} = \begin{bmatrix} 0 \\ b_2 \\ 0 \\ b_4 \end{bmatrix}.$$

- Rows 1 and 3: $f_1 = \dot{x}$ and $f_3 = \dot{\theta}$, so a single 1 in each row.
- Columns 1 and 2 of rows 2 and 4: zero, because f_2, f_4 do not contain x, $\dot{x}$.
- Rows 1 and 3 of $\mathbf{B}$: zero, because f_1, f_3 do not contain F.

Only the six entries in rose need a derivative. The next slide computes them.

Step 3: the six derivatives

A shortcut for $f_2 = N/D$. The quotient rule gives $\partial f_2 = (\partial N \cdot D - N \cdot \partial D)/D^2$. At the equilibrium $N = 0$, so the second term disappears, and with $D = M$ (because $\sin\pi = 0$): $\partial f_2 = \partial N/M$. We only differentiate the numerator, and then put $\theta = \pi$, $\dot\theta = 0$, $F = 0$, $\cos\pi = -1$, $\sin\pi = 0$.

Row 2 (f_2):

$$a_{23} = \frac{1}{M}\frac{\partial N}{\partial \theta} = \frac{1}{M}\, mg\big(\cos^2\theta - \sin^2\theta\big)\Big|_\star = \frac{mg}{M}, \qquad a_{24} = \frac{1}{M}\frac{\partial N}{\partial \dot\theta} = \frac{1}{M}\,\frac{b}{\ell}\cos\theta\Big|_\star = -\frac{b}{M\ell}, \qquad b_2 = \frac{1}{M}\frac{\partial N}{\partial F} = \frac{1}{M}.$$

(The terms with $\dot\theta^2$ or $\dot\theta$ times $\sin\theta$ vanish at the equilibrium.)

Row 4 (f_4): differentiate $f_4 = -\frac{1}{\ell}\big(\cos\theta\, f_2 + g\sin\theta + \frac{b}{m\ell}\dot\theta\big)$, using $f_2 = 0$, $\cos\theta = -1$ at the equilibrium and the row-2 results:

$$\begin{aligned}
a_{43} &= -\frac{1}{\ell}\Big(-\sin\theta\, f_2 + \cos\theta\, a_{23} + g\cos\theta\Big)\Big|_\star = \frac{1}{\ell}\Big(\frac{mg}{M} + g\Big) = \frac{(M+m)\,g}{M\ell},\\
a_{44} &= -\frac{1}{\ell}\Big(\cos\theta\, a_{24} + \frac{b}{m\ell}\Big)\Big|_\star = -\frac{1}{\ell}\Big(\frac{b}{M\ell} + \frac{b}{m\ell}\Big) = -\frac{(M+m)\,b}{Mm\ell^2},\\
b_4 &= -\frac{1}{\ell}\,\cos\theta\, b_2\Big|_\star = \frac{1}{M\ell}.
\end{aligned}$$

Step 4: the matrices A, B, and what they tell us

Collecting the six derivatives, the linearized model is $\delta\dot{\mathbf{x}} = \mathbf{A}\,\delta\mathbf{x} + \mathbf{B}F$, with $\delta\mathbf{x} = (x, \dot{x}, \delta\theta, \delta\dot{\theta})$ and $\delta\theta = \theta - \pi$. With our numbers:

$$\mathbf{A} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & \frac{mg}{M} & -\frac{b}{M\ell} \\ 0 & 0 & 0 & 1 \\ 0 & 0 & \frac{(M+m)g}{M\ell} & -\frac{(M+m)b}{Mm\ell^2} \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 4.905 & -0.167 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 49.05 & -1.667 \end{bmatrix}, \qquad \mathbf{B} = \begin{bmatrix} 0 \\ \frac{1}{M} \\ 0 \\ \frac{1}{M\ell} \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 3.333 \end{bmatrix}.$$

Stability without control. The eigenvalues of $\mathbf{A}$ are

$$0, \quad 0, \quad +6.22, \quad -7.89.$$

The eigenvalue $+6.22$ means the pendulum falls, as expected. The two zeros belong to the cart: nothing pulls it back to $x = 0$.

Controllability (Session 2). The matrix $\mathcal{C} = \begin{bmatrix} \mathbf{B} & \mathbf{AB} & \mathbf{A}^2\mathbf{B} & \mathbf{A}^3\mathbf{B} \end{bmatrix}$ has rank 4 (checked with `np.linalg.matrix_rank`, as in Lab 2). So, even though F acts only on the cart, we can place all four eigenvalues where we want.

In `cartpole.py`, mark `[CP-LIN]`: `linear_model()` builds exactly these $\mathbf{A}$ and $\mathbf{B}$.

Step 5: designing the gain

The controller uses all four states (Session 2):

$$F = -\mathbf{K}\,\delta\mathbf{x} = -k_x\,x - k_{\dot{x}}\,\dot{x} - k_\theta\,\delta\theta - k_{\dot{\theta}}\,\delta\dot{\theta}.$$

Choosing the poles. We ask for four real, negative eigenvalues, -2, -3, -4, -5. The slowest is -2: its term e^{-2t} is below 2 % after about $4/2 = 2\,\mathrm{s}$ (since $e^{-4} \approx 0.02$), so the loop settles in about 2 s.

Computing K. With two states we matched the characteristic polynomial by hand. With four, the algebra is long, so we let the computer do the same matching:

```
K = place_poles(A, B, [-2, -3, -4, -5]).gain_matrix
```

$$\mathbf{K} = \begin{bmatrix} -3.67 & -4.83 & 38.73 & 5.15 \end{bmatrix} \quad\Longrightarrow\quad F = 3.67\,x + 4.83\,\dot{x} - 38.73\,\delta\theta - 5.15\,\delta\dot{\theta}.$$

The next slide explains what these four numbers do.

In `cartpole.py`, mark `[CP-DESIGN]`: `design()`, one call to `place_poles`.

Step 5 (continued): reading the signs of K

Written out, the controller is

$$F = \underbrace{3.67\,x + 4.83\,\dot{x}}_{\text{cart terms}} \underbrace{- 38.73\,\delta\theta - 5.15\,\delta\dot{\theta}}_{\text{pendulum terms}}.$$

Recall that $\delta\theta < 0$ means the pendulum leans to the right (as in the drawing), and $F > 0$ pushes the cart to the right.

- The large angle gain moves the cart *toward the side where the pendulum is falling*, to get back under it: exactly what your hand does with a broom.
- The cart gains look "wrong": a cart to the right ($x > 0$) gets a push to the right. This is deliberate. Pushing the cart further right tips the pendulum to the *left*; then the angle gain makes the cart follow the pendulum, and the cart travels back left. You cannot walk a broom to the left without first tilting it to the left.

The lesson

With an underactuated system (fewer motors than things to control), a good controller sometimes moves the wrong way first. Placing the poles finds those signs for us; intuition alone would not.

Exercise 4

The cart is at rest at $x = 0$ and the pendulum starts at $\theta = \pi - 0.2$, that is $\delta\theta = -0.2$ (leaning to the right, as in the drawing).

1. Compute the force F at $t = 0$ with $\mathbf{K} = \begin{bmatrix} -3.67 & -4.83 & 38.73 & 5.15 \end{bmatrix}$.
2. In which direction does the cart start to move? Does that make sense?
3. The motor can give at most 10 N. Is the first push within that limit?

Hint

At $t = 0$ only one state is not zero.

Exercise 4: solution

Step 1. The state at $t = 0$ is $\delta\mathbf{x} = (0,\ 0,\ -0.2,\ 0)$: only $\delta\theta$ is not zero.

Step 2. Force:

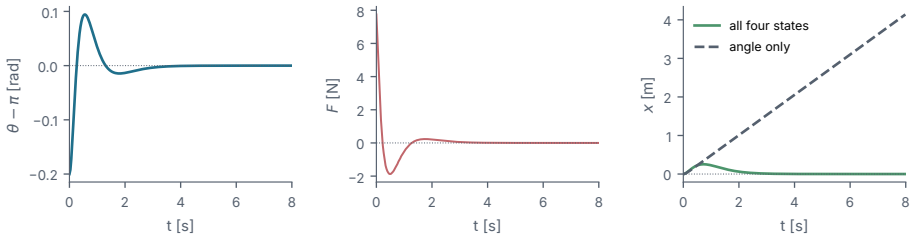
$$F = -\mathbf{K}\,\delta\mathbf{x} = -(38.73)(-0.2) = +7.75\,\text{N}.$$

Step 3. Direction. $F > 0$, so the cart is pushed to the **right**. The pendulum leans to the right, so the cart moves under it, like your hand under a falling broom. Yes, it makes sense.

Step 4. Limit. $7.75\,\text{N} < 10\,\text{N}$: the first push fits. A larger initial tilt asks for more: with $\delta\theta = -0.6$ the formula gives 23 N, the motor saturates, and the simulation shows the pendulum falls (scenario `'far_start'`). As in Session 3, the linear design only works in a neighbourhood of the top.

Step 6: test on the nonlinear cart-pole

The gain designed on the linear model, applied to the nonlinear model of Step 1, starting from $\delta\theta = -0.2$:



- **Angle** (left): back to the top in about 2 s, as the poles promised.
- **Force** (center): starts at 7.75 N (Exercise 4), then changes sign to stop the cart.
- **Cart** (right, green): it travels 0.25 m to the right to catch the pendulum, then returns to $x = 0$.
- **Angle only** (right, dashed): if we set $k_x = k_{\dot{x}} = 0$, the pendulum still stays up, but the cart drifts away, more than 4 m in 8 s. Nothing in the controller knows where the cart is. This is why we feed back *all four* states.

In `cartpole.py`, mark `[CP-DESIGN]`: run `'design'`, then `'angle_only'`.

The three questions of today, on the cart-pole

Everything we learned with the pendulum carries over. Each row is a scenario of `cartpole.py`, and every number was obtained by running it.

Question	Scenario	What happens
wrong model	`'heavy_pole'`	pendulum 50 % heavier: still balances, same speed
too far from the top	`'far_start'`	$\delta\theta = -0.6$: the 10 N motor saturates and the pendulum falls; -0.5 still works
constant push	`'push_cart'`	0.5 N on the cart: balanced, but the cart stays at $x = -0.136$ m, an offset
integral action	`'push_integral'`	add $z = \int x\,dt$ as a fifth state and place five poles: the cart returns to $x = 0$
noisy sensors	`'noise_raw'`	velocities from noisy encoders: the force jitters by about 6.6 N
filter	`'noise_filtered'`	$\tau = 0.05$ s: jitter down to about 1.5 N, still balanced

What we cannot do yet. Bring the pendulum up from the bottom (*swing-up*): the linear design only works near the top. Swing-up is solved with energy, the language of Lyapunov: Module 2.

In `cartpole.py`, mark `[CP-NOISE]`: `controller()` estimates both velocities and filters them, exactly as `velocity()` did in `pendulum_inverted.py`.

Module 1 in one table

Problem	What you see	Today's quick fix	The proper tool, later
wrong model m, ℓ, b not exact	slower response, or the pendulum does not get back up	enough gain margin, k_1 well above $mg\ell$	robustness analysis with Lyapunov (Module 2), online adaptation (Module 3)
constant push $d \neq 0$	a permanent offset	integral action	disturbance estimation (Module 3)
noisy sensor $\eta \neq 0$	the motor torque jitters	low-pass filter on $\dot{\theta}$	observers (Module 3)

The quick fixes work, and engineers use them every day. But each one was found by trial and simulation, and each one comes with a number we picked by hand: a margin, a k_i, a τ.

The rest of the course replaces trial with guarantees: Module 2 tells us when a controller works *without* simulating every case, and Module 3 lets the controller estimate what it cannot measure.

Lab 4: running your own scenario

The lab uses the same file, `pendulum_inverted.py`. You only add lines to the list `SCENARIOS`; you never edit the functions.

Step 1. At the end of `SCENARIOS` there is an example line. Copy it and change the name and the values, one line per case you want to run:

```
    'A1_m075':  dict(m=0.75),
    'A2_d03':   dict(d=0.3),
    'A3_ki10':  dict(d=0.2, ki=10.0),
```

Step 2. Write that name in `SCENARIO = 'A1_m075'` and run the file.

Step 3. Collect the outputs of every run (the animation also plays, to watch what happens):

- a printed line, for example `A1_m075:  final error -0.000 rad, torque jitter after 2 s 0.00 N.m`;
- an image `A1_m075.png` with $\theta - \pi$ and u against time (in Colab, open the folder icon on the left to download it);

Key	Meaning	Default	Where on the slides
`m`, `k1`	real mass [kg], proportional gain	0.5, 3.0915	Question 1
`d`, `ki`	push [N·m] from $t = 1$ s, integral gain	0, 0	Question 2
`sigma`, `ts`, `tau`	noise [rad], sample time [s], filter [s]	0, 0.01, off	Question 3

Lab 4: what to do

Part A: wrong model and constant push

1. Run $m = 0.5,\ 0.75,\ 1.0,\ 1.1,\ 1.5$ kg. Make a table with, for each mass: $mg\ell$, the printed final error, and *returns / does not return*. Does every row agree with the rule $k_1 > mg\ell$?
2. Run $d = 0.1,\ 0.2,\ 0.3$ N·m (with $k_i = 0$). Make a table with the printed final error next to the formula $d/(k_1 - mg\ell)$. Do they agree?
3. Run $d = 0.2$ with $k_i = 1,\ 3,\ 10$. Include the three images. Which k_i would you choose, and why? (One or two sentences.)

Part B: noisy sensor

4. Run `'noise_raw'` and `'noise_filtered'`. Report both printed torque jitters. By what factor does the filter reduce it?
5. Add a scenario with `sigma=0.01, ts=0.005` and no `tau`. Compare its torque jitter with `'noise_raw'` ($T_s = 0.01$). The formula $2\sigma_\eta / T_s$ predicts a factor of 2. What factor do you get?
6. Run `'noise_slow_filter'`. Using its image, describe in two sentences what happens to the pendulum and why.

Lab 4 is Homework 4. Every answer is a table, a number printed by the program, an image saved by the program, or one or two sentences. No new code is needed.

Wrap-up and next session

Today

- a wrong mass is tolerated while $k_1 > mg\ell$;
- a constant push leaves an offset $d/(k_1 - mg\ell)$, and integral action removes it;
- differentiating a noisy signal amplifies the noise by $2/T_s$; a low-pass filter helps, at the cost of delay;
- the cart-pole puts it all together: linearize, check controllability, place four poles, test on the nonlinear model.

Next session: Module 2 begins

- regulation and tracking: holding a point versus following a path;
- the tracking error, and why control design is really about making it go to zero;
- a first nonlinear controller that does not rely on staying near the top.

Homework 4: due before the next session

Complete items 1 to 6 of Lab 4 and submit a **single PDF** containing the requested plots, tables and brief answers. Upload the PDF to the *Homework 4* assignment on Blackboard by the posted deadline. Submissions by email will not be accepted.